General

- 两大类数据结构
  - 线性
    - 一坨数据都有且只有零或一个前驱(前一个元素)和一个后继(后一个元素)
    - e.g. 数组(顺序储存)、链表(链式储存)、栈堆、队列
  - 非线性
    - 多对多的结构
    - e.g. 树、图
- 常用的缓存淘汰策略
  - LRU算法
    - 用双链表维护顺序和保存缓存值，用哈希表快速查询
    - 对于每一个被使用的数据，都移动到双链表的头，表示最近使用，这样尾部就是最少使用的了
    - 哈希表则是用来快速访问缓存值的，这样不用遍历双链表，只是注意这个哈希表没有保存顺序信息的功能
- Complexity:
  - only nested loop makes more complexity. multiple separate loops are still in O(n)
  - 但也得考虑进入和退出的次数，例如滑动窗口其实是O(2n)=O(n)

经验

- Methdology:
  - 第一眼：
    - 枚举所有可能性：很多时候应该从枚举所有可能性开始思考
    - 手做一遍（？）：手做一遍，发掘其中一些规律
  - 第二眼：
    - result信息：留意题目要求的输出，有时候某些信息**不一定需要被保留**（eg 求最大子数组和不需要子数组索引）
    - 反转主次：从原本的 '从 A 推出 B' 改成 '从 B 推出 A' 可能有助解题（柱状图内的矩阵 就是很好的例子）
    - 转化问题：把这个问题转化为另一个等价但描述不同的问题，可能可以简化思考过程
    - 关键字：可能有用的**关键字**：相邻/连续、最大最小、以此开头/结尾
  - 最后几眼（逻辑已经完备，需要最终优化）：
    - 排序：如果可以预见这道题的复杂度必然很高，**排序**是非常好用的优化方法
    - 预处理：预处理以简化主循环的复杂度
    - 反转主次：从原本的 '从 A 推出 B' 改成 '从 B 推出 A' 可能有助优化（字符串匹配 pattern 和柱状图内的矩阵 就是很好的例子）
    - 遍历融合：把原本需要两个遍历的逻辑合到一个遍历内
    - 以**反向**的方式计算答案，最后反过来输出，可能会比正向算答案更有机会成功。（详见210）
- 代码：
  - `const count = map.get(s) ?? 0; map.set(s, count+1)` 可以很方便地更新频率map
  - set/map to array：Array.from

- HackerRank
  - 性能要求极度严格，一点点多余的代码都会导致 case 失败
  - HackerRank 的超时和错误答案的展示方式是一样的

优化的经验

- 排序 和 break
- 反转主次
  - 遍历线性结构中的可能性时不要一位一位扫，直接按固定单位长度来遍历，这个单位长度可以是 patterns 每个元素的长度
  - 嵌套 loop 时，尝试把内外层反过来，可能更方便剪枝

数组

- 子串是连续的，子序列/子集不是

dp

- 定义：
  - 设计概念
    1. how to define dp[i]?
    2. how to update dp[i+1] from dp[i]?
    3. how to get the result from dp?
    4. 自底向上（递推），自顶向下（暴力递归+备忘录 + 剪枝）
- General 经验：
  - some abstract mindset:
    - how to keep transfering the state? (states can be +ve or -ve states that benefit or harm the result respectively)
  - 有些 dp 的更新是相邻的，有些是可以隔好几个的。后者一般和直接枚举目标值有关。（eg 零钱问题，单词拼接问题）
- 具体案例经验
  - 常见用法：
    - 前缀和
    - 暴力递归
    - 递归
    - 备忘录
  - 最长递增子序列
  - 路径压缩
  - 背包问题：
    - 定义：
      - 一般问 ‘存不存在一种组合符合某个条件’ 就是背包问题
    - dp[i][w] 的定义：对于前 i 个物品，当前背包的容量为 w，这种情况下可以装的最大价值是 dp[i][w]。
    - 经验：

        - 为什么能够遍历所有可能性？因为遍历时会把每个物品都在每个容量下进行 dp 判断，所以无论哪个物品在什么容量下，都会有对应的结果
            - 每个 dp[i][w] 都是考虑了 nums[i-1] 是否装进来之后的最优解，也就是 dp[i-1][w] 和 dp[i-1][w-nums[i]]，也因此每次往回看结果都是考虑了装或者不装的完整结果。
            - 有时候装了的结果和不装是一样的，有时候又是不同的，这造成了巨量的 dynamic ，也因此能解决问题
            - 同一个位置的值是一样的，但含义包含非常多可能性，比如可以是
                - 前 i 个物品的结果
                - 自己装进去的结果
                - base case 的结果
    - 有好几种类型的背包问题，它们的处理方法相当不同
        - eg 背包最大值问题
            - 放进去和不放进去的 dynamic 存在于：1. 容量是否够；2. 放进去（找前 [i-1][w-nums[i]] 个的结果合在一起）和不放进去（只取前 [i-1][w] 的结果）哪个最终值更大
        - eg 背包能否塞满问题
            - 放进去和不放进去的 dynamic 存在于：1. 容量是否够；2. 放进去（找前 [i-1][w-nums[i]] 个的结果合在一起）和不放进去（只取前 [i-1][w] 的结果）哪个结果是 true，或者都为 false
        -

moving windows & double pointer

- 一些概念
    1. when to move left and right?
    2. when to stop moving left and right?
    3. how to update the result?
    4. 关键不在于怎么停在最优的left和right，而在于怎么遍历所有有利可能性
- 其实移动窗口就是一种双指针
- 一般有两种启动方式
    1. left = right = begin;
    2. left = begin, right = end;
- 算法框架
    - eg ```ts function sliceWindow(s: string) { // 1. 窗口内需要持续维护的状态。可能是 map，可能是 number，按题目来就行 let window = ...; // 2. across 每个窗口都需要尝试更新的题目答案 let result = ...;

```
    // 3. 循环所需的左右指针，可能是左 0 右 0 也可能是左 0 右 尾，这里先讨论都是 0 的情况
    // 注意默认是左闭右开，方便处理 edge case
    let left = ..., right = ...;
    while (right < s.length) {
        // 4. 每次循环都有几件事要做
        //     a. 右指针右移
```

```
        ++right;
        //     b. 更新 window 状态和 result
        append element to window;
        update result;
        //     c. 判断需不需要左移指针
        while (needShrink) {
            //     d. 更新 window 状态和看情况更新 result
            shift element from window;
            update result;
            ++left;
        }

    }

    return result;
}
```

linked list

- typical questions:
  - nth node from the end.
  - half from the end.
  - dummy list
  - intersection (linked list concatentation) (可以理解为对齐一定相同的尾部)
    - 其实就是因为从交叉点开始两条list会变得一样，因此拼接对齐之后的尾部就正好会是交叉点了
- dummy node can be used as the head node of modified linked list (so even if it is the original first node that changed, the dummy node is still the head)
- 单链表：delete ==> 2 ptrs(被删除的那个，接上的前一个). swap ==> 3 ptrs(被swap的俩，接上的前一个)
- recursion is useful but costly. iteration is space saving but harder to implement.

tree

- 定义：
  - 二叉搜索树 Binary Search Tree (BST)
    - 二叉查找/搜索/排序树
    - 中序遍历是升序的数组
    - 有序数组构建二叉搜索树会变成链表
  - 平衡/完全二叉树 Balanced/Complete Binary Tree
    - 每个节点的左右子树高度差不超过1
    - 优先填满左边，有空位只会是右边有连续的空位
    - 会使用旋转来平衡
      - 因此中序遍历不会有得出链表的情况出现

    - 平衡/完全二叉搜索/查找/排序树
        - 常见实现：
            - AVL 树
                - 用旋转来维护
            - 红黑树
                - 用颜色来维护
            - B 树
                - 通过节点分裂和合并来保持树的平衡。
            - Splay 树
                - 通过将最近访问的节点旋转到根节点来保持树的平衡
    - 完美二叉树
        - 节点全满的二叉树
    - 前序是进入，后序是退出，中序是退出与进入中间
        - ↙→，→↖，↗↘
- General 经验：
    - 分解问题为 pipeline 形式，然后每个节点都处理所有问题的一个 stage
    - 遍历思路：前序；分解问题思路：后序
    - 前序能利用参数，后序能利用参数和返回值；对于需要递归检查某种约束，前序和后序都有可取之处：前序可以传递参数，后序可以获得子最优解 （***递归与递推的异同***）
    - 如果要储存所有叶子节点，空间复杂度是O(n)（O(2^(round_down(log(n)) + 1)) = O(n+1) = O(n)
    - 有用的公式：
        - 出度为0的节点数 = 出度为2的节点数 + 1
        - 满二叉树的总结点数量 = 叶子节点数量 * 2 - 1
        - 总节点数为奇数时没有度为1的节点，为偶数时只会有一个度为1的节点
- 具体案例经验：
    - BFS: 用queue， DFS: 用stack

graph theory

- 本质上就是交叉的树，可以用标记来模拟成每个节点最多四个子节点的树
- DFS和BFS在这里很重要
    - DFS本质上就是递归
    - BFS本质上就是队列
- 有向边，无向边
    - 名流问题
- 图的储存形式
    - general：G = {V, G}
    - 邻接表：类似多叉树的数组形式
        - `graph[from] = <Array_of_to>;`
        - 邻接表的DFS昭示着依赖的排序 (可能需要倒序罢了，就看邻接表的边代表着依赖还是被依赖)
        - 参考207.ts
    - 邻接矩阵
        - n x n的矩阵，1表示i和j有边、0表示没有
        - 有向边和无向边就只是矩阵是否对称的区别
- 入度、出度

    - 也就是每个节点被多少节点依赖，又依赖多少节点
- 环检测：
    - DFS：onPathTrack + visited，加上遍历过程中的卫语句就行
        - visited用来剪枝，onPathTrack用来入栈出栈和判断环
    - BFS：
        - 把所有一开始入度为0的节点入队，开始遍历
        - 出队当前节点，对其每一个子节点减少入度，并判断子节点入度是不是变成0，为0的话入队
        - 最后队列为空跳出循环了，如果有环，必然有某些节点没被遍历完（因为环节点的入度不为0），判断方法就是遍历节点时维护count， count和总节点数不一致就代表有环
- 拓扑排序：
    - 拉平一个图，且使其没有任何右依赖左的排序
    - 因此有环是无法拓扑排序的
    - DFS：
        - 为什么邻接表的DFS和拓扑排序有关？
            1. 因为能够往下遍历的都是非环的，遍历到了叶子节点自然就是一组符合拓扑排序的排列
            2. 至于会不会出现两条依赖关系都有相同叶子节点，导致根节点的选择不同产生不同的拓扑排序，这是可能的，但是不要紧，因为拓扑排序对同一张图完全可以有不同的结果，只要保证没有右依赖左就行。从不同的根节点先开始遍历不会破坏拓扑排序
    - BFS：
        - 找入度为0的节点不断入队遍历就完成了
    - 参考210.ts
- 并查集算法 (Union-Find Algorithm)
    - 用来实现对联通分量的检测的
    - 实现四个方法：find、union、isConnected、getGroups
    - 优化方法：路径压缩、按秩合并(较小的树接到较大的树下)
    - eg

    ```
    class UF {
        constructor(n: number) {}
    }
    ```

- 路径压缩：和dp、动态规划有关系
- 最大团、极大团搜索算法

queue

- FIFO
- .push(), .pop(), .front(), .back(), .empty()
- priority queue:
    - implemented by heap, therefore have priority.
- 注意，BFS一个tree的时候，queue如果用的两层，不可以只判断外层的长度，因为内层必定会有一个array实例，哪怕是空的

stack

- s.top(); s.pop(); s.push();
- **单调栈**
  - 和单调队列一样，重点是在优先队列之外提供一个复杂度更低的解法。
  - 但单调栈或者队列会抛弃部分节点，而优先队列会保留全部节点

recursion

- base case: 除了 是否null，还可以试试看用 a是否＞b（比如在a或b会被绝对数值增减的情况下）
- 大框架
  - 终止模块
  - 递归模块
    - 数据获取模块（前序遍历时储存所需数据）
- 重要的是思考每次recursion可以获得什么参数和将要返回什么参数，不要给脑子堆栈
- 还得留意参数ref问题
- 回溯算法
  - 在递归模块中，除了前序获取元素，还要后序删除元素

字符串

- General：
  - 字符串对比很多时候都得 m*n 或者 n*2，也就是遍历所有可能性，很少能优化的
  - 经常会有图/多叉树遍历的情况，因为要计算各种排列
- 具体案例：
  - 字符串对比：（具体可以看 1143.最长公共子序列）
    - 相同时两边同时步进，不同时 dp 传递 state
  - 编辑距离：双指针，多叉树遍历

排序相关

- 二分查找
  - 框架

```
function binarySearch<T>(arr: T[], target: T): number {
    let left = 0;
    let right = INIT_RIGHT; // arr.length, arr.length - 1
    while ( CONDITION ) {   // left < right, left <= right
        // 在while主要操作就可以避免很多奇怪变体出问题
        let mid = left + ROUND_BEHAVIOUR((right - left)/2);  // 防止溢出
        if ( arr[mid] === target ) {
            FOUND_BEHAVIOUR;           // return mid, right = mid
        } else if ( arr[mid] < target ) {
            left = NEXT_LEFT;
```

```
        } else if ( arr[mid] > target ) {
            right = NEXT_RIGHT;   // mid, mid - 1
        }
    }
    RETURN_BEHAVIOUR;   // return left/right/-1/奇怪的condition啥的
}
```

- 有相当多变体，包括：找一个数，找左边界，找右边界
- 通用的逻辑：
  - 注意，二分查找不要只局限于mid和target的对比，只要可以分为固定的常数级sub case，就可以使用二分查找思路（详见33.ts
  - floor和ceil影响mid最后会和l重叠还是和r重叠，也影响left和mid或者right和mid之间要不要=，floor就是left和mid对比要，ceil就是right和mid要
  - 无论单数还是双数最后都至多会塌陷到对比最后两个数，这样想方便处理edge case
  - FOUND_BEHAVIOUR能控制最终得到左一-1还是右一+1
  - 左闭右闭和<=绑定
  - =mid+1可能越界，而单纯=mid不会越界
- 找target：左闭右闭
  - INIT_RIGHT = arr.length - 1; CONDITION = left <= right; ROUND_BEHAVIOUR = floor; FOUND_BEHAVIOUR = return mid; NEXT_LEFT = mid + 1; NEXT_RIGHT = mid - 1; RETURN_BEHAVIOUR = mid
  - condition为 <= 的原因：
    - 剩最后两个数时，会忽略掉target和left或者right重叠的情况。eg：lm重叠目标在r 或者 rm重叠目标在l， < 会跳过这两个cases
    - 数学来讲，考虑闭区间[a, b]， a和b相同时这个区间并不为空，如果 < 就会跳过了这个可能性
  - 终结时left和right一定是错开的
  - 没有target时，无法控制在"空隙"左边还是右边。
- 找左边界（左一）：左闭右开
  - INIT_RIGHT = arr.length; CONDITION = left < right; ROUND_BEHAVIOUR = floor; FOUND_BEHAVIOUR = right = mid; NEXT_LEFT = mid - 1; NEXT_RIGHT = mid; RETURN_BEHAVIOUR = left;
  - 原理：
    - r不用动时，l和mid在动，mid会不断判断有没有更左的target以更新r，反之亦然，因此能找到左一
    - 返回的是left，不是mid，且CONDITION是</终结条件是l=r，最终返回的是l和r重叠的idx，而r一定是左一
    - RETURN_BEHAVIOUR也可以返回right
  - 如果target不存在，返回大于target的第一个元素
    - 因为FOUND和NEXT_RIGHT是一样的
    - mid < target是l=mid+1，但 > target就只是r=mid没有+1，因此必定会被这个mid+1推到"空隙"右边再终结
  - 左闭右闭的版本：
    - INIT_RIGHT = arr.length-1; CONDITION = left <= right; FOUND_BEHAVIOUR = right = mid + 1; NEXT_LEFT = mid - 1; NEXT_RIGHT = mid + 1; RETURN_BEHAVIOUR = left;

            - 但RETURN_BEHAVIOUR不能right，只能是left；
    - 找右边界（右一）
        - 和左边界的FOUND_BEHAVIOUR、ROUND_BEHAVIOUR、RETURN_BEHAVIOUR相反就行了，当然结果也是相反的
    - 没有target时返回-1
        - 记得做边界检查
        - RETURN_BEHAVIOUR做if判断就行，如果等于target就返回，否则返回-1
- 耐心排序
    - 把所有元素按照一定规则分为多个组：组间有排序；一个组内的元素必然能从其左边的组中找到<=该元素的元素。
    - 依次拿牌分组，如果从左到右第一个某一堆最上边的牌大于本牌则压上去，否则创建新组
    - 可以解决 **最长递增子序列** 问题
    - 感觉和图论有点关系

原地算法

- 基础做法：
    - use the original/part of array to store the result. then O(1) space complexity
    - use a single extra variable to store the result. then O(1) space complexity
- 原地翻转
    - 原地翻转一个句子的单词，不反转字母（没太明白为什么要用这种方法，直接暴力点不行吗）
        - 把句子的所有字母翻转
        - 再把每个单词单独内部反转
    - 原地轮转数组：有很多方法
        1. 把整个数组翻转过来，再把前n位和剩下的位各自分开再翻转
        2. 环形替换 (力扣189)

环相关

- 轮转数组，用 (idx + k) mod n， k是轮转的位数，n是数组长度
- % 可以用来 range control (189.cpp 轮转数组)

矩阵相关

- 当你需要思考矩阵遍历时，想想往不同方向时有什么不同含义（参考 240 搜索二维矩阵）
    - 矩阵居然可以和二叉树联动（240题，从右上角往左下看类似二叉排序树）
- 经验：
    - 矩阵很容易 induce 成别的题目，也就是题目转换

回文

- 记住，双数和单数的子串分开处理就能把整串对比给简化成仅对比新字符
- 可以把左右区间映射到二维的dp中来实现O(1)的回文判断

回溯

- 路径前进和后退
- 剪枝
  - 剪枝是最 hack 的，比如 79 的剪枝可以翻转单词，用更低频的字符开始往下，以更高概率发生剪枝
- 可以思考一下**动态规划**和**路径压缩思路**
- 排序好数组可以方便往下走时去重剪枝，对于字符串每一位的往下走在回溯时还能自动获得字典序

去重

- 空间换时间：用Set和Map，`f(x) = <cache_result>`
- visited数组：
  - 多维度 visited 可用于多个参数的去重，`f(x, y) = <cache_result>`
    - 例如：图遍历/回溯中用多维visited数组储存cache
  - 注意，如果目标的数据是一位的，只是其某些特征可以用二位表示并解决问题，注意处理边界问题，例如用二维数组储存字符串回文结果（详见131）
- 倒三角遍历：`for x in range(k), for x in range(k+1)`
  - 从k开始遍历就行，会去除permutation的重复，也就是获得combination
- 如果涉及考虑重复元素，或者大小比较的情况，可以尝试给原数组排序
  - 这是一个相当有趣的技巧，本质上就是把重复元素出现顺序固定死了，如果重复元素a不出现，b也不能出现，所以只会有a -> b的情况，不会有a -> b和b -> a同时出现的重复情况。所以这本质上是一个倒三角遍历的变异（这个居然没有例子？）
  - 而且把目标先排序了的话，回溯的顺序刚好就是字典序
  - `if ( i > 0 && nums[i] == nums[i-1] && exist[i-1] == 0 ) {...}`
- 第一个不重复的和最后一个不重复的有细微的差别

深度优化

- 找出一些其实不会影响complexity，但逻辑上会减少运算的优化方法
  - 比如反转遍历顺序
- 创建数组是费时的操作。如果想要创建坐标数组，可以用index表示y，item表示x，极致压缩。

很白痴的索引问题

- ‘闭’ 是指包含，‘开’是指不包含
- 左闭右开就是常见的取前不取后
- 注意反向遍历时，终结条件是`>= 0`

other

- i++ or ++i:
  - i++: return i, then i = i + 1

    - ++i: i = i + 1, then return i

哈希冲突

- 开放寻址法
    - 线性探测（Linear Probing）
    - 二次探测（Quadratic Probing）
    - 双重哈希（Double Hashing）
    - 随机探测（Random Probing）
- 哈希表操作复杂度
    - 插入/删除/查询：
        - 平均情况：O(1)（假设哈希函数均匀分布，负载因子适中）。
        - 最坏情况：O(n)（所有键冲突，退化为链表）。
    - 负载因子（α, Load Factor）：
        - 定义：负载因子 = 元素数量/哈希表大小
        - 通常需保持α ≤ 0.7（线性探测）、α ≤ 0.5（二次/双重哈希）、α < 0.9（链地址法）以避免性能下降。

经验

- 最 hack 的两种题目是回溯和 dp ，真的很可怕
- 创建多维数组的最快方法：

```
function create2DArray(rows, cols) {
    const arr = new Array(rows);
    for (let i = 0; i < rows; i++) {
        arr[i] = new Array(cols).fill(0);
    }
    return arr;
}
```

- 子串遍历可以用滑动窗口
- 进一步优化：滚动变量/数组代替整个变量/数组进行迭代

to_learn:

- 前序中序后序遍历的迭代实现和Morris遍历
- 二叉树、红黑树、avl树
- 1382.cpp的原地算法
- 105索引问题还没想明白
- dp 找一道做一做
- tree 层序遍历，各种遍历方法

- 146: LRU还没写完
- 300: 耐心排序
- 218：skyline

other: 148 skipped 437 skipped

常见题目

- 二叉树
  - 记住前序中序后序就行